

50Days-coding-challenge

Day-40[SQL]

Packaging Domain

Name:M NITHIN

Problem Statement

You are working as a **Data Analyst in a packaging company** that supplies boxes, bottles, and wrappers to different clients.

The company wants insights into:

- Production efficiency
- Defect rates
- Revenue performance

Tables

1. products

column type

product_id INT product_name VARCHAR

category VARCHAR (Box, Bottle,

Wrapper) cost_per_unit DECIMAL

2. production

column type

production_id INT product_id

INT

production_date	DATE
quantity_produced	INT
defective_quantity	INT

sales

column	type
sale_id	INT
product_id	INT
sale_date	DATE
quantity_sold	INT
selling_price_per_unit	DECIMAL

Tasks

Task 1: Defect Rate per Product

Write a query to calculate: $\text{defect_rate} = \frac{\text{defective_quantity}}{\text{quantity_produced}}$

Return:

- product_name
- total_produced
- total_defective
- defect_rate

The screenshot shows a PostgreSQL query editor with a query that calculates the defect rate percentage for various products. The query is as follows:

```

SELECT
  p.product_name,
  SUM(pr.quantity_produced) AS total_produced,
  SUM(pr.defective_quantity) AS total_defective,
  ROUND(
    SUM(pr.defective_quantity) * 100.0 /
    SUM(pr.quantity_produced), 2
  ) AS defect_rate_percentage
FROM products p
JOIN production pr

```

The query results are displayed in a table with the following columns: product_name, total_produced, total_defective, and defect_rate_percentage. The results are as follows:

product_name	total_produced	total_defective	defect_rate_percentage
Gift Wrapper	3100	155	5.00
Small Box	2100	90	4.29
Large Box	1250	65	5.20
Medium Box	1700	45	2.65
Glass Bottle	1450	30	2.07
Food Wrapper	4100	220	5.37
Plastic Bottle	2500	120	4.80

Task 2: Top Performing Product (Revenue) Find

the top product by total revenue.

revenue = quantity_sold * selling_price_per_unit

The screenshot shows a PostgreSQL query editor with a query that finds the top performing product by total revenue. The query is as follows:

```

SELECT
  p.product_name,
  SUM(s.quantity_sold * s.selling_price_per_unit) AS total_revenue
FROM products p
JOIN sales s
ON p.product_id = s.product_id
GROUP BY p.product_name
ORDER BY total_revenue DESC
LIMIT 1;

```

The query results are displayed in a table with the following columns: product_name and total_revenue. The results are as follows:

product_name	total_revenue
Medium Box	34100.00

Task 3: Monthly Production Trend

Show:

- month

- total production

Sort by month.

The screenshot shows a PostgreSQL query editor with the following SQL query:

```
SELECT
  TO_CHAR(production_date, 'YYYY-MM') AS month,
  SUM(quantity_produced) AS total_production
FROM production
GROUP BY TO_CHAR(production_date, 'YYYY-MM')
ORDER BY month;
```

The query results are displayed in a table with 2 rows:

month	total_production
2026-01	7800
2026-02	8400

A status message at the bottom indicates: "Successfully run. Total query runtime: 160 msec. 2 rows affected."

Task 4: Category-wise Profit

profit = selling_price - cost_price

Return:

- category
- total revenue
- total cost
- total profit

The screenshot shows a PostgreSQL query editor with the following SQL query:

```
SELECT
  p.category,
  SUM(s.quantity_sold * s.selling_price_per_unit) AS total_revenue,
  SUM(s.quantity_sold * p.cost_per_unit) AS total_cost,
  SUM(
    (s.selling_price_per_unit - p.cost_per_unit)
    * s.quantity_sold
  ) AS total_profit
FROM products p
```

The query results are displayed in a table with 3 rows:

category	total_revenue	total_cost	total_profit
Box	95600.00	64250.00	31350.00
Bottle	49500.00	33000.00	16500.00
Wrapper	39300.00	16300.00	23000.00

A status message at the bottom indicates: "Total rows: 3 Query complete 00:00:00.205"

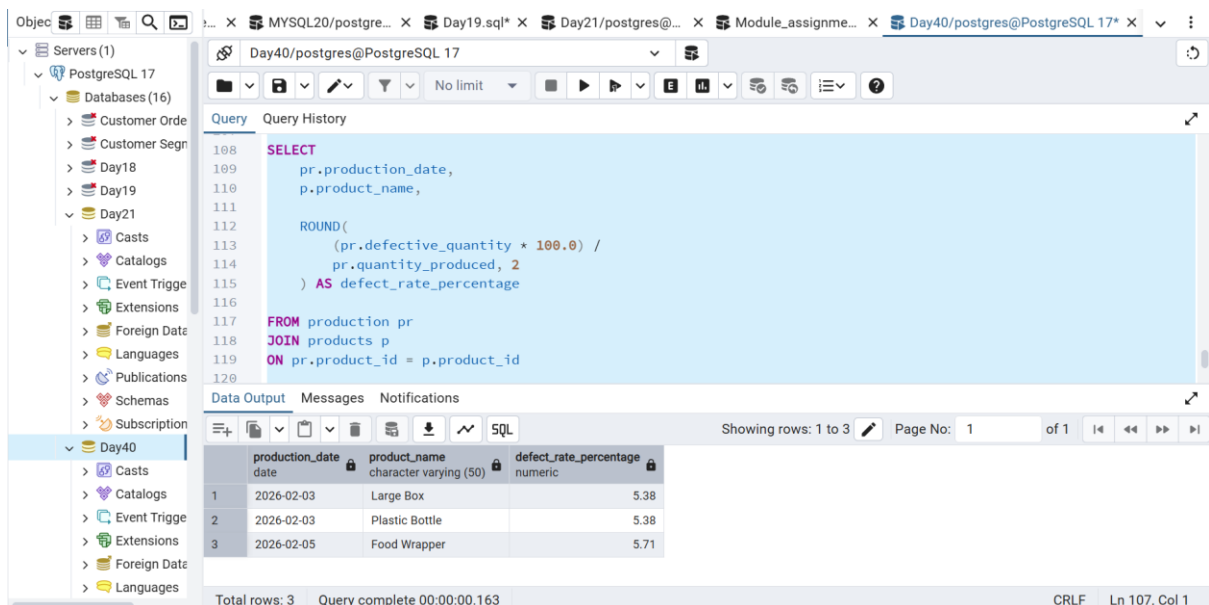
Task 5: Identify High Defect Days

Find days where:

defect_rate > 5%

Return:

- date
- product_name
- defect_rate



The screenshot shows a PostgreSQL query editor interface. The query is as follows:

```
SELECT
    pr.production_date,
    p.product_name,
    ROUND(
        (pr.defective_quantity * 100.0) /
        pr.quantity_produced, 2
    ) AS defect_rate_percentage
FROM production pr
JOIN products p
ON pr.product_id = p.product_id
```

The query results are displayed in a table with the following data:

	production_date	product_name	defect_rate_percentage
1	2026-02-03	Large Box	5.38
2	2026-02-03	Plastic Bottle	5.38
3	2026-02-05	Food Wrapper	5.71

The interface also shows a sidebar with a database tree, a query history panel, and a status bar at the bottom indicating 'Total rows: 3' and 'Query complete 00:00:00.163'.

Task 6: Running Total Revenue

Calculate cumulative revenue over time.

Servers (1)
PostgreSQL 17
Databases (16)
Customer Order
Customer Segn
Day18
Day19
Day21
Casts
Catalogs
Event Trigg
Extensions
Foreign Date
Languages
Publications
Schemas
Subscription
Day40
Casts
Catalogs
Event Trigg
Extensions
Foreign Date
Languages

Day40/postgres@PostgreSQL 17

Query Query History

```

145 SELECT
146   sale_date,
147
148   SUM(quantity_sold * selling_price_per_unit) AS daily_revenue,
149
150   SUM(
151     SUM(quantity_sold * selling_price_per_unit)
152   ) OVER (
153     ORDER BY sale_date
154   ) AS running_total_revenue
155 FROM sales
156
157

```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No: 1 of 1

	sale_date date	daily_revenue numeric	running_total_revenue numeric
1	2026-01-05	13500.00	13500.00
2	2026-01-06	15400.00	28900.00
3	2026-01-07	15000.00	43900.00
4	2026-01-08	12000.00	55900.00

Total rows: 14 Query complete 00:00:00.226 CRLF Ln 145, Col 1

Task 7: Best Category by Profit Margin

$\text{profit_margin} = \text{profit} / \text{revenue}$

Return top category.

Servers (1)
Query Tool Workspace
Databases (16)
Customer Order
Customer Segn
Day18
Day19
Day21
Casts
Catalogs
Event Trigg
Extensions
Foreign Date
Languages
Publications
Schemas
Subscription
Day40
Casts
Catalogs
Event Trigg
Extensions
Foreign Date
Languages

Day40/postgres@PostgreSQL 17

Query Query History

```

161 SELECT
162   p.category,
163
164   ROUND (
165     (
166       SUM(
167         (s.selling_price_per_unit - p.cost_per_unit)
168         * s.quantity_sold
169       )
170     ) /
171     SUM(s.quantity_sold * s.selling_price_per_unit)
172     * 100,
173

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	category character varying (20)	profit_margin_percentage numeric
1	Wrapper	58.52

Total rows: 1 Query complete 00:00:00.202 CRLF Ln 161, Col 1

Task 8: Unsold Inventory

Find products where:
produced > sold

The screenshot shows a PostgreSQL query editor interface. On the left, a tree view displays the database structure, including 'Servers(1)', 'PostgreSQL 17', and various databases like 'Customer Order', 'Customer Segn', 'Day18', 'Day19', 'Day21', 'Casts', 'Catalogs', 'Event Trigg', 'Extensions', 'Foreign Date', 'Languages', 'Publications', 'Schemas', 'Subscription', and 'Day40'. The 'Day40' database is selected, showing its contents: 'Casts', 'Catalogs', 'Event Trigg', 'Extensions', 'Foreign Date', and 'Languages'.

The main query editor displays the following SQL query:

```

SELECT
  p.product_name,
  SUM(pr.quantity_produced) AS total_produced,
  SUM(s.quantity_sold) AS total_sold,
  (
    SUM(pr.quantity_produced)
    - SUM(s.quantity_sold)
  ) AS unsold_inventory
FROM products p

```

Below the query editor, the 'Data Output' tab shows the results of the query. The results are displayed in a table with 5 columns: 'product_name', 'total_produced', 'total_sold', and 'unsold_inventory'. The table contains 4 rows of data:

	product_name	total_produced	total_sold	unsold_inventory
1	Gift Wrapper	6200	5800	400
2	Small Box	4200	3800	400
3	Large Box	2500	2200	300
4	Medium Box	3400	3100	300

The status bar at the bottom indicates 'Total rows: 7', 'Query complete 00:00:00.174', and 'CRLF Ln 210, Col 54'.

Table Creation

```

CREATE TABLE products (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(50),
  category VARCHAR(20),
  cost_per_unit DECIMAL(10,2)
);

```

```

CREATE TABLE production (
  production_id INT PRIMARY KEY,
  product_id INT,
  production_date DATE,
  quantity_produced INT,
  defective_quantity INT,

  FOREIGN KEY (product_id) REFERENCES products(product_id)
);

```

```

CREATE TABLE sales (
  sale_id INT PRIMARY KEY,
  product_id INT,
  sale_date DATE,
  quantity_sold INT,

  selling_price_per_unit DECIMAL(10,2),

  FOREIGN KEY (product_id) REFERENCES products(product_id)
);

```

);

Insert Data – products

INSERT INTO products VALUES

(1, 'Small Box', 'Box', 10.00),
(2, 'Medium Box', 'Box', 15.00),
(3, 'Large Box', 'Box', 20.00),
(4, 'Plastic Bottle', 'Bottle', 8.00),
(5, 'Glass Bottle', 'Bottle', 12.00),
(6, 'Food Wrapper', 'Wrapper', 2.00),
(7, 'Gift Wrapper', 'Wrapper', 3.00);

Insert Data – production

INSERT INTO production VALUES

(1, 1, '2026-01-01', 1000, 50),
(2, 2, '2026-01-01', 800, 20),
(3, 3, '2026-01-02', 600, 30),
(4, 4, '2026-01-02', 1200, 60),
(5, 5, '2026-01-03', 700, 10),
(6, 6, '2026-01-03', 2000, 100),
(7, 7, '2026-01-04', 1500, 75),
(8, 1, '2026-02-01', 1100, 40),
(9, 2, '2026-02-02', 900, 25),
(10, 3, '2026-02-03', 650, 35),
(11, 4, '2026-02-03', 1300, 70),
(12, 5, '2026-02-04', 750, 20),
(13, 6, '2026-02-05', 2100, 120),
(14, 7, '2026-02-06', 1600, 80);

Insert Data – sales

```
INSERT INTO sales VALUES
```

```
(1, 1, '2026-01-05', 900, 15.00),
```

```
(2, 2, '2026-01-06', 700, 22.00),
```

```
(3, 3, '2026-01-07', 500, 30.00),
```

```
(4, 4, '2026-01-08', 1000, 12.00),
```

```
(5, 5, '2026-01-09', 650, 18.00),
```

```
(6, 6, '2026-01-10', 1800, 5.00),
```

```
(7, 7, '2026-01-11', 1400, 7.00),
```

```
(8, 1, '2026-02-05', 1000, 15.00),
```

```
(9, 2, '2026-02-06', 850, 22.00),
```

```
(10, 3, '2026-02-07', 600, 30.00),
```

```
(11, 4, '2026-02-08', 1100, 12.00), (12, 5, '2026-02-09', 700, 18.00),
```

```
(13, 6, '2026-02-10', 2000, 5.00), (14, 7, '2026-02-11', 1500, 7.00);
```